

Assignment: LLM Inference Systems

1 Why is Inference Memory Bandwidth-Bound Important?

Memory bandwidth refers to the amount of data that can be transferred between GPU memory (VRAM) and GPU processing cores per second. It is typically measured in GB/s and plays a crucial role in Large Language Model (LLM) inference because model parameters are stored in memory and must be continuously accessed during token generation.

Large Language Models contain billions of parameters. During inference, the model generates output one token at a time. For each token, the GPU repeatedly reads model weights and Key-Value (KV) cache data from memory. Although modern GPUs can perform trillions of computations per second, they often spend more time waiting for data to arrive from memory than actually performing calculations.

This phenomenon makes LLM inference a memory bandwidth-bound workload. In other words, inference speed is limited by how quickly data can move between memory and compute units rather than by raw computational power.

The relationship can be approximated as:

$$\text{Tokens per Second} \propto \frac{\text{Memory Bandwidth}}{\text{Model Size}} \quad (1)$$

Higher memory bandwidth provides:

- Faster token generation
- Lower latency
- Better throughput
- Improved user experience
- Higher concurrent user capacity

For example:

GPU	Memory Bandwidth
RTX 4090	~1008 GB/s
NVIDIA H100 SXM	~3350 GB/s

The H100 serves large models significantly faster because it can transfer data more efficiently.

Memory bandwidth also becomes increasingly important with longer context windows because the KV cache grows and requires additional memory transfers.

Modern inference frameworks address bandwidth limitations through:

- Quantization (INT8, INT4)
- Flash Attention
- Paged Attention
- KV Cache Optimization

- Tensor Parallelism

Therefore, memory bandwidth is one of the most important hardware characteristics when deploying large language models because it directly affects performance, scalability, and operational cost.

2 How Can We Calculate the Overall Memory Required to Store a Model? How Are LLM Models Represented While Storing?

Large Language Models learn patterns during training and store this knowledge as numerical values called parameters or weights. These parameters are stored in tensors and occupy memory depending on the numerical precision used.

The memory required to store a model can be calculated using:

$$\text{Model Memory} = \text{Number of Parameters} \times \text{Bytes per Parameter} \quad (2)$$

Common data types include:

Data Type	Bytes per Parameter
FP32	4 Bytes
FP16	2 Bytes
BF16	2 Bytes
INT8	1 Byte
INT4	0.5 Byte

Example: 7 Billion Parameter Model

$$\text{FP32} \quad 7B \times 4 = 28GB \quad (3)$$

$$\text{FP16} \quad 7B \times 2 = 14GB \quad (4)$$

$$\text{INT8} \quad 7B \times 1 = 7GB \quad (5)$$

$$\text{INT4} \quad 7B \times 0.5 = 3.5GB \quad (6)$$

However, inference requires more than just model weights.

Total memory usage can be estimated as:

$$\text{Total Memory} = \text{Weights} + \text{KV Cache} + \text{Activations} + \text{Framework Overhead} \quad (7)$$

Components

Model Weights — Learned parameters of the model.

KV Cache — Stores information about previous tokens; required for efficient attention computation.

Activations — Intermediate outputs generated during inference.

Framework Overhead — Memory consumed by inference engines such as vLLM or TensorRT.

KV Cache size can be approximated using:

$$KV\ Cache = 2 \times Layers \times Hidden\ Size \times Sequence\ Length \times Batch\ Size \times Bytes \quad (8)$$

How LLMs Are Stored

LLMs are stored as tensors (multidimensional arrays).

Examples:

```
Wq = Tensor[4096][4096]
Wk = Tensor[4096][4096]
Wv = Tensor[4096][4096]
```

Important stored tensors include:

- Embedding Matrix
- Query Weights
- Key Weights
- Value Weights
- Output Projection Weights
- Feed Forward Weights
- Layer Normalization Parameters

Popular storage formats:

Format	Usage
PyTorch .bin	Standard training checkpoint
Safetensors	Secure and fast loading
GGUF	Local inference
ONNX	Cross-platform deployment
TensorRT Engine	NVIDIA optimized deployment

These files contain billions of numerical values representing the intelligence learned during training.

3 Explain Each Layer of Inferencing and Its Importance to an Institution/Company

Modern LLM applications consist of multiple layers working together to provide AI services. A user prompt passes through several components before a response is generated.

3.1 User Interface Layer

This is the point where users interact with the system.

Examples: ChatGPT, Mobile Apps, Internal Enterprise Portals

Responsibilities: Accept user prompts, display responses, collect feedback.

Importance: A well-designed interface improves user adoption and overall experience.

3.2 API Gateway Layer

Acts as the entry point for all requests.

Examples: FastAPI, Kong, NGINX

Responsibilities: Authentication, authorization, request validation, rate limiting.

Importance: Protects enterprise AI systems from misuse and unauthorized access.

3.3 Orchestration Layer

Coordinates workflows between models, tools, and external systems.

Examples: LangChain, LangGraph, LlamaIndex

Responsibilities: Prompt construction, workflow execution, tool integration, multi-step reasoning.

Importance: Allows AI systems to perform complex business tasks.

3.4 Retrieval Layer (RAG)

Provides external knowledge to the model.

Components: Embedding Models, Vector Databases

Examples: Pinecone, Weaviate, Qdrant

Responsibilities: Store enterprise knowledge, retrieve relevant documents, reduce hallucinations.

Importance: Allows companies to use private documents and internal knowledge bases.

3.5 Inference Engine Layer

Executes the model efficiently.

Examples: vLLM, TensorRT-LLM, LMDeploy, Ollama

Responsibilities: Model loading, memory optimization, KV cache management, request scheduling.

Importance: Directly affects latency, throughput, and cost.

3.6 Model Layer

Contains the actual Large Language Model.

Examples: Llama, Gemma, Qwen, Mistral

Responsibilities: Reasoning, language understanding, text generation.

Importance: Determines response quality and intelligence.

3.7 Hardware Layer

Provides computational resources.

Examples: RTX 4090, H100, MI300X

Responsibilities: Execute computations, store model weights, manage memory.

Importance: Determines scalability and deployment cost.

3.8 Monitoring Layer

Tracks system health.

Examples: Prometheus, Grafana

Responsibilities: Monitor latency, track GPU utilization, detect failures.

Importance: Ensures reliability and performance in production environments.

4 Pick Any Two GPUs, Compare Their Costs, and Estimate Maximum Model Size

GPU selection significantly impacts deployment cost, performance, and scalability.

4.1 GPU 1: NVIDIA RTX 4090

Specifications

Metric	Value
VRAM	24 GB
Memory Type	GDDR6X
Bandwidth	~1008 GB/s
CUDA Cores	16,384
Approx Rental Cost	\$0.50–0.80/hr

Maximum Model Size

FP16

$$24GB \div 2 = 12B \quad (\approx 12 \text{ Billion Parameters}) \quad (9)$$

INT4

$$24GB \div 0.5 = 48B \quad (\approx 48 \text{ Billion Parameters}) \quad (10)$$

Suitable for: Development, fine-tuning, small-scale inference.

4.2 GPU 2: NVIDIA H100 SXM

Specifications

Metric	Value
VRAM	80 GB
Memory Type	HBM3
Bandwidth	~3350 GB/s
CUDA Cores	16,896
Approx Rental Cost	\$2.50–4/hr

Maximum Model Size

FP16

$$80GB \div 2 = 40B \quad (\approx 40 \text{ Billion Parameters}) \quad (11)$$

INT4

$$80GB \div 0.5 = 160B \quad (\approx 160 \text{ Billion Parameters}) \quad (12)$$

Suitable for: Enterprise AI, high-concurrency serving, large context windows.

4.3 Comparison

Feature	RTX 4090	H100
VRAM	24 GB	80 GB
Bandwidth	1008 GB/s	3350 GB/s
FP16 Capacity	~12B	~40B
INT4 Capacity	~48B	~160B
Cost	Low	High
Enterprise Usage	Moderate	Excellent

The H100 offers significantly better inference performance due to its larger VRAM and higher memory bandwidth.

5 Research Two Serving Frameworks and Explain Why You Would Choose Them

Serving frameworks provide the runtime environment required to deploy and manage LLMs efficiently.

5.1 Framework 1: Ollama

Ollama is a lightweight framework designed for local deployment of large language models.

Supported Models: Llama, Gemma, Qwen, Mistral

```
ollama run llama3
```

Advantages

- Simple installation
- Beginner-friendly
- Runs locally
- Supports GGUF models
- Good privacy

Limitations

- Limited enterprise features
- Lower throughput than specialized serving engines

Use Cases

- Personal projects
- Development
- Local experimentation
- Offline AI applications

5.2 Framework 2: LMDeploy

LMDeploy is a high-performance serving framework developed by OpenMMLab.

Key Features: TurboMind backend, quantization support, high throughput, efficient memory usage.

Advantages

- Fast inference
- Production-ready
- Better GPU utilization
- Supports large models

Limitations

- More complex setup than Ollama
- Requires GPU optimization knowledge

Use Cases

- Enterprise deployments
- Multi-user applications
- Production APIs

5.3 Comparison

Feature	Ollama	LMDeploy
Ease of Use	Excellent	Moderate
Throughput	Medium	High
Production Ready	Limited	Yes
Local Deployment	Excellent	Good
Enterprise Usage	Moderate	Excellent

Ollama is ideal for rapid experimentation, while LMDeploy is more suitable for large-scale production deployments.

6 Run NVIDIA-SMI and Explain the Output

NVIDIA-SMI (System Management Interface) is a command-line utility provided by NVIDIA for monitoring GPU hardware and resource utilization.

Developers use NVIDIA-SMI to:

- Monitor GPU usage
- Check memory consumption
- Verify CUDA installation
- Diagnose performance issues

Command:

```
nvidia-smi
```

Example Output:

```
GPU Name: RTX 4090
Driver Version: 560.xx
CUDA Version: 12.6
Memory Usage: 12GB / 24GB
GPU Utilization: 92%
Temperature: 67C
Power Usage: 310W / 450W
Processes: python.exe
```

GPU Name

Identifies the installed GPU. Example: RTX 4090. Used to determine hardware capabilities.

Driver Version

Example: 560.xx. The GPU driver enables communication between the operating system and GPU.

CUDA Version

Example: 12.6. Indicates the supported CUDA toolkit version. Many AI frameworks require specific CUDA versions.

Memory Usage

Example: 12GB / 24GB. Shows current VRAM consumption. Important for determining whether a model can fit into memory.

GPU Utilization

Example: 92%. Represents how actively the GPU is being used. High utilization generally indicates efficient resource usage.

Temperature

Example: 67°C. Shows GPU thermal status. Very high temperatures may cause performance throttling.

Power Consumption

Example: 310W / 450W. Indicates current power draw relative to maximum capacity. Useful for infrastructure planning.

Running Processes

Example: `python.exe`. Displays applications currently using the GPU. Useful for identifying resource-intensive processes.